



Institute of Information Technology

Jahangirnagar University, Savar, Dhaka-1342

Lab Report - 06

Lab Title: Introduction to Shell Scripting

Course Code: ICT-3102

Course Title: Operating System Lab

Submitted by:

Md. Shaon Khan

Id: 1984

Batch: 52 Batch

Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor, Institute of Information Technology
Jahangirnagar University

Date: May 5, 2026

1. Objective

The objective of this laboratory session is to introduce students to the fundamentals of shell scripting in a Unix/Linux environment using the Bash interpreter. By the end of this session, students are expected to:

- Create and execute shell scripts using vim, and manage permissions with chmod.
- Declare and use user-defined variables, and access built-in environment variables (\$HOME, \$SHELL).
- Produce formatted output using echo, echo -e, echo -n, and printf.
- Capture dynamic runtime data via command substitution \$(command).
- Accept interactive user input using read.
- Implement conditional logic with if, if-else, if-elif-else, and case statements.
- Perform iteration using for (list and C-style), while, and until loops, with break and continue for flow control.
- Perform integer arithmetic using \$(()) expansion with +, -, *, /, %, and **.
- Define reusable functions, pass arguments via \$1/\$2, and use return with \$? to retrieve exit status.

2. Theory

A shell script is a plain text file containing a sequence of commands that the Bash interpreter executes sequentially. Shell scripting is fundamental to Unix/Linux system administration, allowing repetitive tasks to be automated and packaged into reusable executable programs.

Shebang and Permissions: The first line `#!/bin/bash` (shebang) instructs the OS kernel to use `/bin/bash` as the interpreter. `chmod +x` grants the execute bit required for `./execution`; `sh script.sh` bypasses this requirement by invoking the interpreter directly.

Variables and Output: Variables are assigned without spaces (`name="Shaon"`) and referenced with `$ ($name)`. Environment variables (`$HOME`, `$SHELL`) are pre-set by the OS. `echo` prints to stdout; `echo -e` interprets escape sequences; `echo -n` suppresses the trailing newline; `printf` provides C-style format specifiers for precise alignment.

Conditionals: `if-then-elif-else-fi` structures evaluate conditions in `[ ]`. Numeric operators: `-eq`, `-ne`, `-gt`, `-ge`, `-lt`, `-le`. The `case-esac` statement matches a variable against explicit patterns terminated with `;;` and uses `*` as a default wildcard.

Loops: `for` iterates over lists or uses C-style arithmetic `(( init; cond; incr ))`. `while` repeats while its condition is true; `until` repeats until its condition becomes true. `break` exits a loop; `continue` skips to the next iteration. Arithmetic expansion `$(( ))` performs native integer arithmetic including `**`, with integer division truncating results.

Functions: Functions are defined as `name() { }` and called by name with arguments. Inside a function, `$1` and `$2` are the function's own positional parameters. `return` sets the exit status code; `?` retrieves it immediately after the call. Return values are limited to 0-255; larger values require `echo` and command substitution.

3. Commands Used

The table below lists every command, operator, and shell construct used throughout this laboratory session.

Command / Construct	Description
#!/bin/bash	Shebang line; specifies the Bash interpreter for the script
vim	Text editor used to create and edit shell scripts
more	Displays file content one screen at a time
chmod +x	Grants execute permission to a shell script file
chmod -x	Removes execute permission from a shell script file
./script.sh	Executes a shell script from the current directory
sh script.sh	Runs a script using sh interpreter; does not need execute bit
ls / ls -l	Lists directory contents; -l shows permissions in detail
echo	Prints text or variable values to standard output
echo -e	Enables interpretation of backslash escape sequences (\n, \t)
echo -n	Prints text without a trailing newline
printf	Formats and prints data using C-style format specifiers
#	Denotes a comment line; ignored during script execution
read	Reads a line of user input from stdin into a variable
\$VARIABLE	References the value stored in a variable
\$HOME	Environment variable: path to the user's home directory
\$SHELL	Environment variable: path of the active shell interpreter
\$(command)	Command substitution: replaces expression with command output
\$(())	Arithmetic expansion for integer arithmetic
if / fi	Conditional block executed when condition is true
if-else / fi	Two-branch conditional block
if-elif-else / fi	Multi-branch conditional block
case / esac	Pattern-matching multi-branch statement
-eq / -ne	Numeric: equal to / not equal to
-gt / -ge	Numeric: greater than / greater than or equal to
-lt / -le	Numeric: less than / less than or equal to
for i in list	Iterates over a list of values
for ((; ;))	C-style numeric for loop with init/condition/increment
while / do / done	Loops while the condition remains true
until / do / done	Loops until the condition becomes true
break	Exits the enclosing loop immediately

continue	Skips current iteration; jumps to next loop cycle
+ - * / %	Arithmetic: add, subtract, multiply, integer divide, modulo
**	Arithmetic: exponentiation
function name()	Defines a reusable named function
\$1, \$2	Positional parameters: function/script arguments
return	Exits a function and sets its return/exit status code
\$?	Special variable: exit status of the last command or function

4. Procedure

We began the lab session from an empty working directory. The terminal prompt used throughout was:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$
```

All scripts were created using the vim text editor, saved with :wq, then made executable with chmod +x before execution. Each subsection below documents the exact script, output, and screenshots from the actual lab session.

4.1 Creating a Shell Script, Shebang Line, and chmod

Purpose: To understand how a shell script is created using vim, how execute permissions are controlled using chmod +x and chmod -x.

Command: vim welcome.sh → chmod +x welcome.sh → ./welcome.sh → sh welcome.sh

Script:

```
echo "Welcome to OS Lab 06"
```

Screenshot:

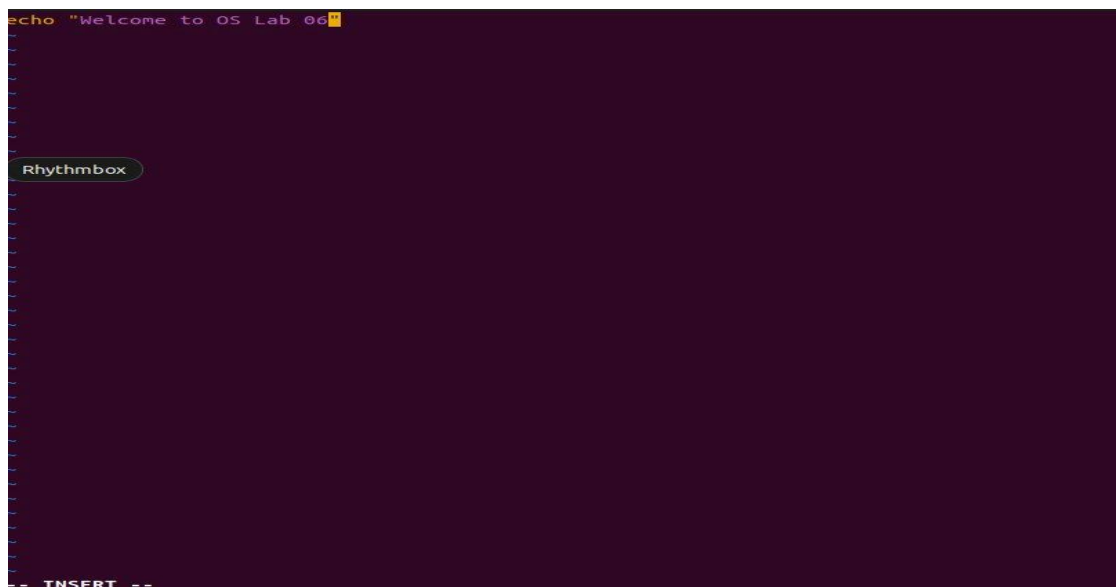


Fig 4.1(a) — vim editor showing welcome.sh content

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim welcome.sh
```

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ more welcome.sh
echo "Welcome to OS Lab 06"
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls -l
total 4
-rwxrwxr-x 1 shaonkhan shaonkhan 28 May 26 13:20 welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod -x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
bash: ./welcome.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ sh welcome.sh
Welcome to OS Lab 06

```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ ls
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ more welcome.sh
echo "Welcome to OS Lab 06"
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ls -l
total 4
-rwxrwxr-x 1 shaonkhan shaonkhan 28 May 26 13:20 welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod -x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
bash: ./welcome.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x welcome.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./welcome.sh
Welcome to OS Lab 06
shaonkhan@shaon:~/OS_Lab/LAB_06$ sh welcome.sh
Welcome to OS Lab 06

```

Fig 4.1(b) — Terminal showing chmod, ./welcome.sh, Permission denied, and sh welcome.sh

Observation: The `ls -l` output confirmed `chmod +x` set the executable bit (`rwxrwxr-x`). Removing permissions with `chmod -x` immediately caused a 'Permission denied' error on `./welcome.sh`, but `sh welcome.sh` still worked because it invokes the interpreter directly without requiring the execute bit. This is the fundamental distinction between the two execution methods.

4.2 Variables

Purpose: To understand how to declare and use user-defined variables in a shell script, assigning a value and referencing it with the `$` prefix.

Command: `vim variable.sh` → `chmod +x variable.sh` → `./variable.sh`

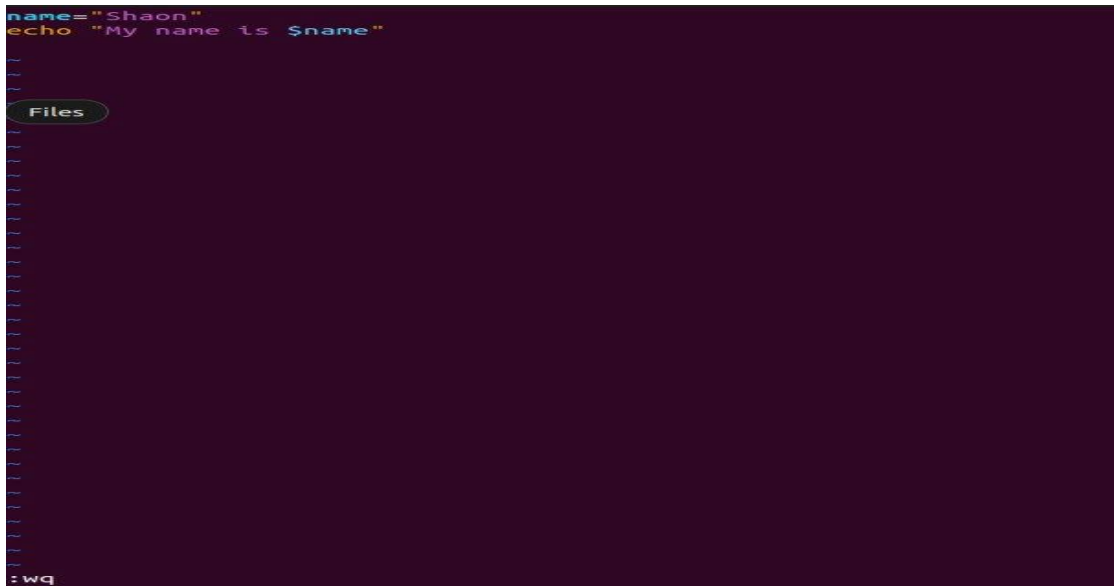
Script:

```

name="Shaon"
echo "My name is $name"

```

Screenshot:



Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim variable.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./variable.sh
bash: ./variable.sh: Permission denied
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x variable.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./variable.sh
My name is Shaon
shaonkhan@shaon:~/OS_Lab/LAB_06$

```

Fig 4.2(c) — Terminal output of variable.sh showing 'My name is Shaon'

Observation: The variable name was assigned the string "Shaon" without spaces around =. Referencing \$name inside double quotes caused Bash to substitute it with its value at runtime. The initial 'Permission denied' error reinforced that chmod +x is a prerequisite for ./execution.

4.3 echo -e and printf (Formatted Output)

Purpose: To demonstrate echo -e interpreting \n escape sequences to produce multi-line output, and printf performing the same with C-style format strings.

Command: vim comment.sh → chmod +x comment.sh → ./comment.sh

Script:

```

echo -e "Line 1\nLine 2\nLine 3"
printf "Line 1\nLine 2\nLine 3\n"

```

Screenshot:

```

echo -e "Line 1\nLine 2\nLine 3"
printf "Line 1\nLine 2\nLine 3\n"

```

Fig 4.3(a) — vim editor showing echo -e and printf in comment.sh

Output:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./comment.sh
Line 1
Line 2
Line 3
Line 1
Line 2
Line 3

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x comment.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./comment.sh
Line 1
Line 2
Line 3
Line 1
Line 2
Line 3

```

Fig 4.3(b) — Terminal showing identical 3-line output from both echo -e and printf

Observation: Both echo -e and printf produced identical three-line output by interpreting \n as a newline character. Plain echo (without -e) would print the literal text '\n'. printf is preferred when precise column widths or numeric formatting are needed.

4.4 Environment Variables (\$HOME, \$SHELL)

Purpose: To explore pre-defined environment variables that the operating system automatically provides to all processes, specifically \$HOME and \$SHELL.

Command: vim env.sh → chmod +x env.sh → ./env.sh

Script:

```

echo "Home directory: $HOME"
echo "Current directory: $SHELL"

```


4.5 Command Substitution (\$(date))

Purpose: To demonstrate command substitution using the \$(command) syntax, which executes a command and captures its output into a variable for use inside the script.

Command: vim date.sh → chmod +x date.sh → ./date.sh

Script:

```
date_today=$(date)

"echo Today is $date_today"
```

Screenshot:

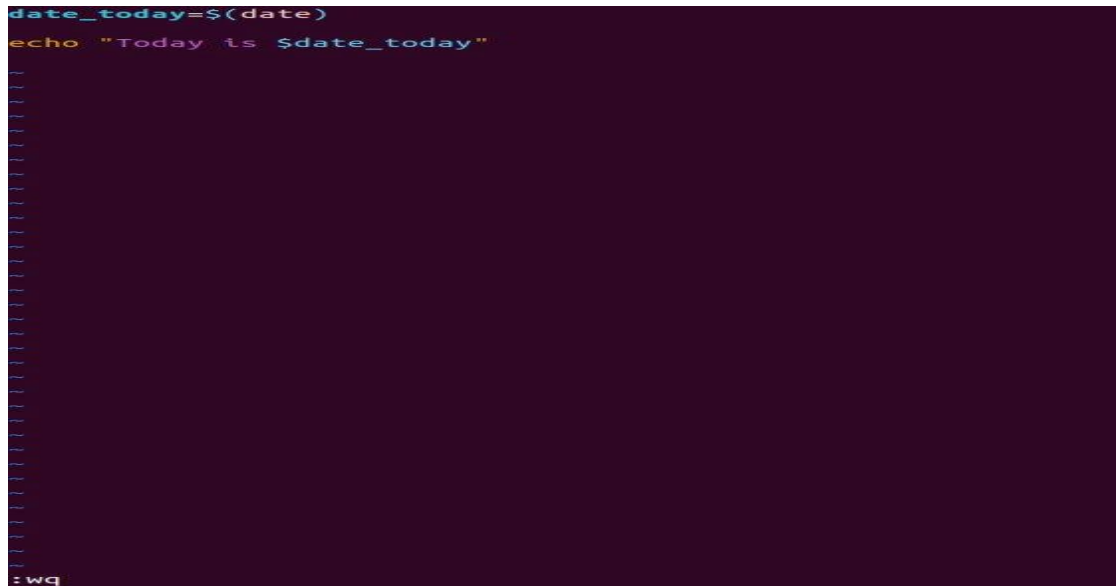


Fig 4.5(a) — vim editor showing date.sh with \$(date) command substitution

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim date.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x date.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./date.sh
Today is Wed 03 Jun 2026 10:28:14 PM +06
```

Screenshot:

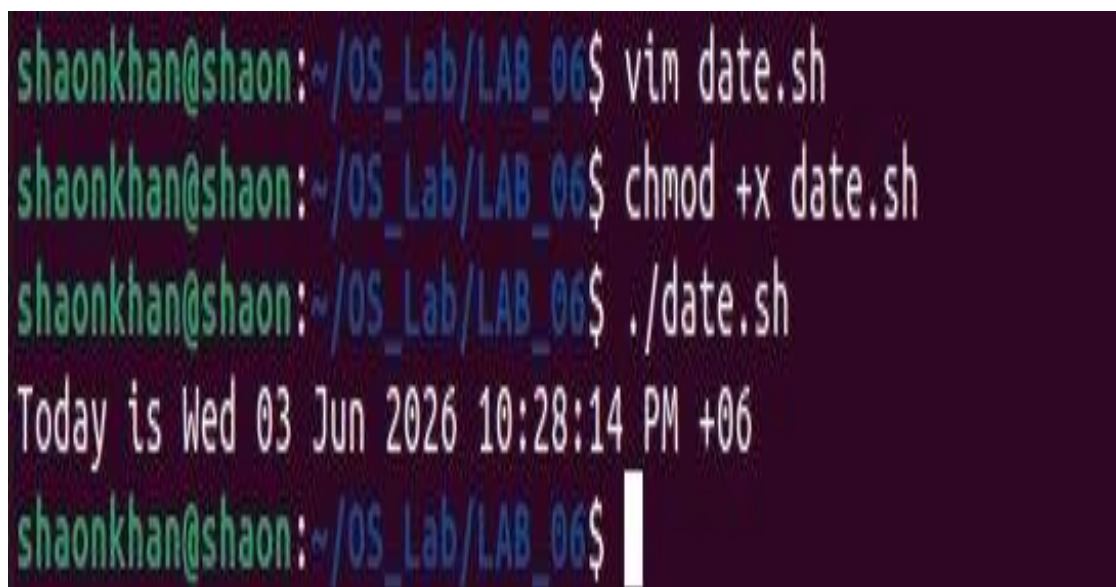


Fig 4.5(b) — Terminal showing live system date captured via command substitution

Observation: \$(date) executed the date command in a subshell and stored its full output — including date, time, and Bangladesh Standard Time offset (+06) — in date_today. This makes scripts dynamic, incorporating live system state without user input.

4.6 Reading User Input with read

Purpose: To demonstrate how read accepts interactive input from the user at runtime and stores it in a variable for use later in the script.

Command: vim input.sh → chmod +x input.sh → ./input.sh

Script:

```
echo "Enter your name:"
read username

echo "Welcome, $username!"
```

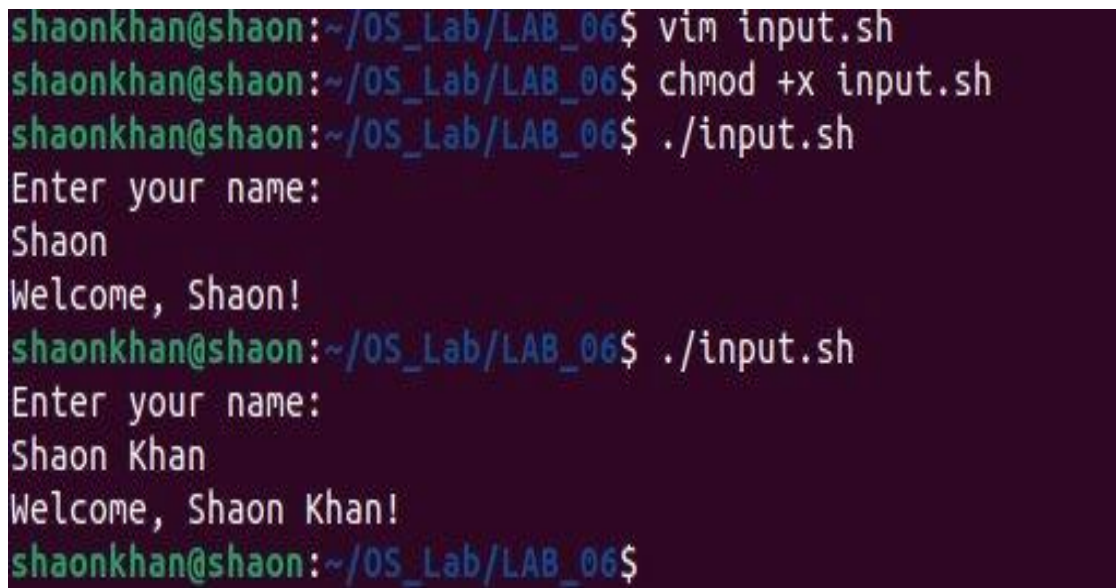
Screenshot:



Fig 4.6(a) — vim editor showing input.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon
Welcome, Shaon!
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon Khan
Welcome, Shaon Khan!
```

Screenshot:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x input.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon
Welcome, Shaon!
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./input.sh
Enter your name:
Shaon Khan
Welcome, Shaon Khan!
shaonkhan@shaon:~/OS_Lab/LAB_06$
```

Fig 4.6(b) — Terminal showing input.sh run twice with 'Shaon' and 'Shaon Khan'

Observation: read pauses execution and waits for the user to type a value and press Enter. Testing with both 'Shaon' and 'Shaon Khan' confirmed that read captures the entire line including spaces into a single variable. This is the foundation of interactive, menu-driven shell programs.

4.7 if Statement

Purpose: To implement a simple if statement that evaluates a numeric condition and executes a block only when the condition is true.

Command: vim if.sh → chmod +x if.sh → ./if.sh

Script:

```
num=10
if [ $num -gt 5 ]
then
    echo "Number is greater than 5"
fi
```

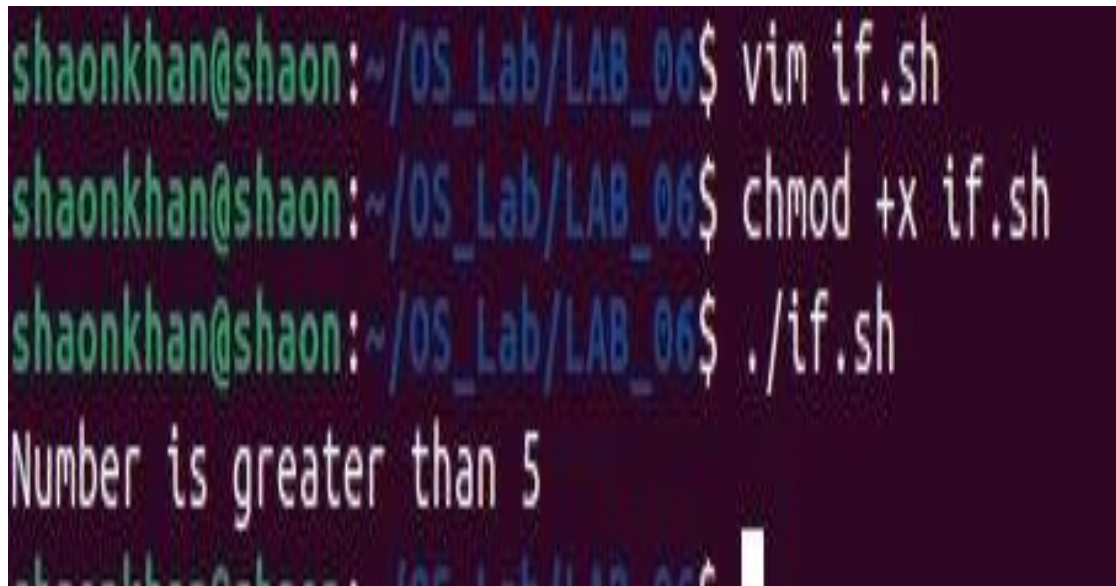
Screenshot:

```
num=10
if [ $num -gt 5 ]
then
    echo "Number is greater than 5"
fi
:~
```

Fig 4.7(a) — vim editor showing if.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./if.sh
Number is greater than 5
```

Screenshot:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x if.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./if.sh
Number is greater than 5
```

Fig 4.7(b) — Terminal output confirming the if condition evaluated true

Observation: With num=10, the condition [\$num -gt 5] evaluated to true and the echo statement executed. The -gt operator performs numeric greater-than comparison. fi closes the if block. If num had been 3, no output would appear since the entire block would be skipped.

4.8 if-else Statement

Purpose: To extend the if construct with an else branch, ensuring exactly one of two blocks always executes based on the condition result.

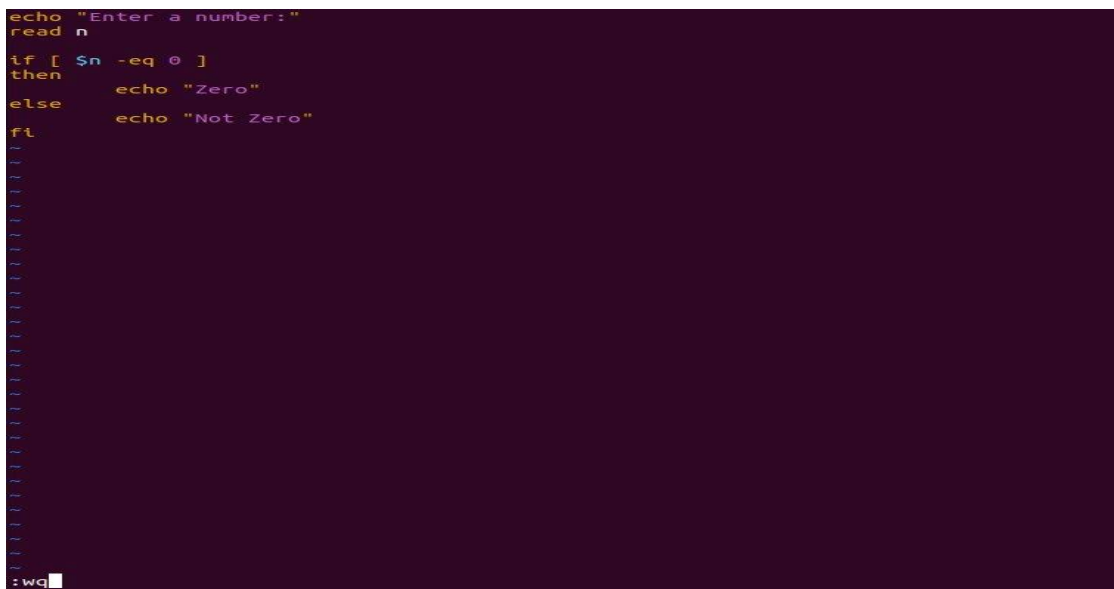
Command: `vim iselse.sh → chmod +x iselse.sh → ./iselse.sh`

Script:

```
echo "Enter a number:"
read n

if [ $n -eq 0 ]
then
    echo "Zero"
else
    echo "Not Zero"
fi
```

Screenshot:



```
echo "Enter a number:"
read n
if [ $n -eq 0 ]
then
    echo "Zero"
else
    echo "Not Zero"
fi
:wq
```

Fig 4.8(a) — vim editor showing iselse.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
5
Not Zero
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
0
Zero
```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x iselse.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
5
Not Zero
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./iselse.sh
Enter a number:
0
Zero

```

Fig 4.8(b) — Terminal showing iselse.sh with inputs 5 (Not Zero) and 0 (Zero)

Observation: Input 5 caused the if condition (`$n -eq 0`) to fail, executing the else branch ('Not Zero'). Input 0 matched the condition, printing 'Zero'. The if-else guarantees that exactly one branch executes in every execution, providing complete binary decision coverage.

4.9 if-elif-else Statement

Purpose: To implement multi-branch conditional logic using if-elif-else, allowing range-based classification where only the first matching condition executes.

Command: `vim grade.sh → ./grade.sh`

Script:

```

echo "Enter your marks:"
read marks

if [ $marks -ge 80 ]
then
    echo "Grade A+"
elif [ $marks -ge 70 ]
then
    echo "Grade A"
elif [ $marks -ge 60 ]
then
    echo "Grade B"
else
    echo "Grade F"
fi

```


4.10 case Statement

Purpose: To implement exact-value pattern matching using the case statement, which is more readable than chained if-elif blocks when matching a variable against a fixed set of options.

Command: `vim choice.sh → chmod +x choice.sh → ./choice.sh`

Script:

```
echo "Enter a choice (y/n):"
read choice

case $choice in
    y) echo "You chose YES" ;;
    n) echo "You chose NO" ;;
    *) echo "Invalid choice" ;;
esac
```

Screenshot:

```
echo "Enter a choice (y/n):"
read choice
case $choice in
    y) echo "You chose YES" ;;
    n) echo "You chose NO" ;;
    *) echo "Invalid choice" ;;
esac
bm:
```

Fig 4.10(a) — vim editor showing case statement in choice.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
y
You chose YES
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
o
Invalid choice
```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x choice.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
y
You chose YES
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./choice.sh
Enter a choice (y/n):
o
Invalid choice

```

Fig 4.10(b) — Terminal showing choice.sh: 'y' matched YES, 'o' fell through to wildcard

Observation: Input 'y' matched the first pattern and printed 'You chose YES'. Input 'o' matched none of the explicit patterns and was caught by the wildcard *) printing 'Invalid choice'. The ;; terminator closes each branch. The * wildcard is equivalent to the default clause in C's switch statement, ensuring no input goes unhandled.

4.11 for Loop (List-Based and C-Style)

Purpose: To demonstrate iteration using the standard Bash for loop (over a list of values) and the C-style for loop using arithmetic expressions for init, condition, and increment.

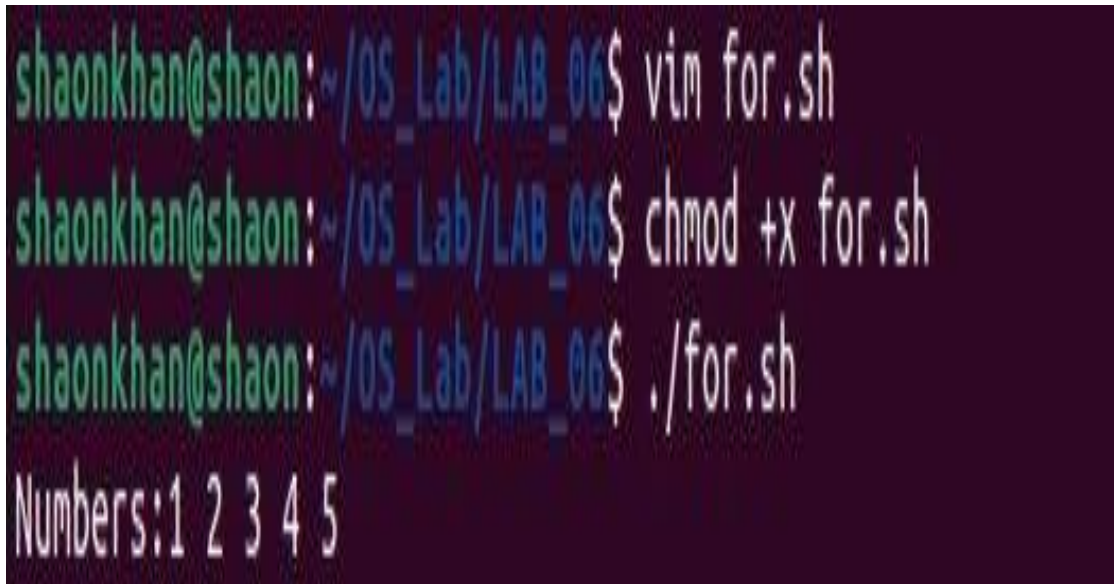
Command: vim for.sh / vim cfor.sh → chmod +x → ./for.sh / ./cfor.sh

Script (for.sh — list-based):

```

echo -n "Numbers:"
for i in 1 2 3 4 5
do
    echo -n "$i "
done
echo

```


Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim for.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x for.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./for.sh
Numbers:1 2 3 4 5

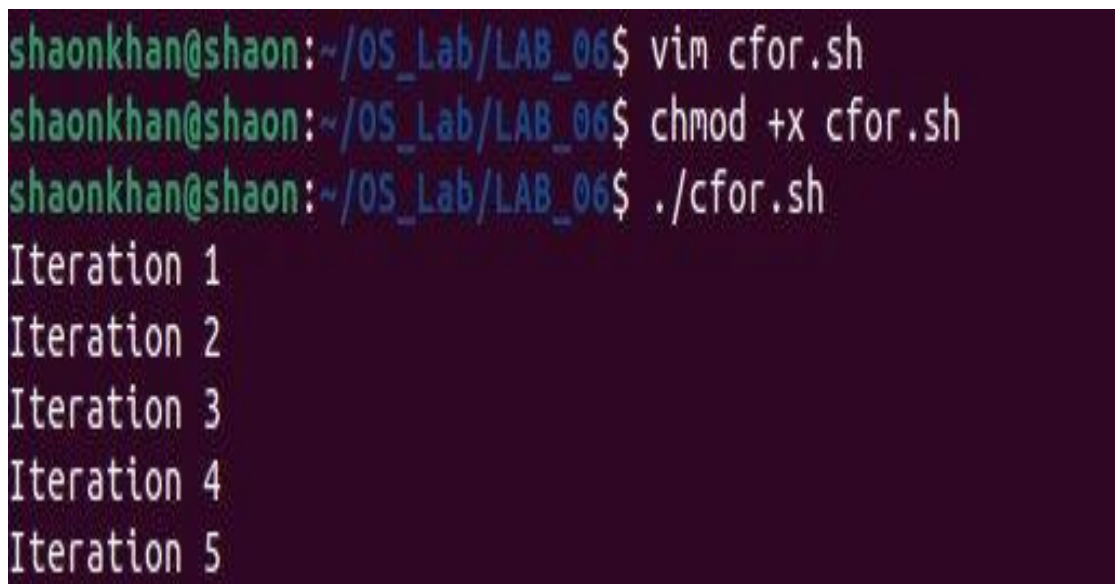
```

Fig 4.11(c) — Terminal output of for.sh printing numbers on a single line

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./cfor.sh
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x cfor.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./cfor.sh
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5

```

Fig 4.11(d) — Terminal output of cfor.sh showing 5 iterations

Observation: The list-based for loop iterated over the literal values 1 2 3 4 5. `echo -n` suppressed newlines so all values appeared on a single line, followed by a bare `echo` to terminate the line. The C-style for loop used arithmetic expressions ((`i=1`; `i<=num`; `i++`)) familiar to C/Java programmers. Both loops iterated exactly 5 times, confirming correct behaviour.

4.12 while Loop

Purpose: To implement a while loop that continues executing as long as a condition remains true, and to observe how a syntax error in the condition is diagnosed and corrected.

Command: `vim while.sh → chmod +x while.sh → ./while.sh`

Script:

```
count=1
while [ $count -le 5 ]
do
    echo "Count = $count"
    count=$((count+1))
done
```

Screenshot:



```
count=1
while [ $count - le 5 ]
do
    echo "Count = $count"
    count=$((count+1))
done
```

Fig 4.12(a) — vim editor showing while.sh

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
./while.sh: line 2: [: syntax error: '-' unexpected
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5
```

Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
./while.sh: line 2: [: syntax error: '-' unexpected
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim while.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./while.sh
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5

```

Fig 4.12(b) — Terminal showing initial syntax error in while.sh, then corrected output

Observation: On first execution, a syntax error on line 2 was reported: '[: syntax error: '-' unexpected'. This was caused by a missing space between [and the condition, or an incorrectly typed operator. After opening vim to fix the condition spacing, the script executed correctly, printing Count = 1 through Count = 5. This practical debugging experience highlighted that Bash is sensitive to spacing inside [] test expressions.

4.13 until Loop

Purpose: To demonstrate the until loop, which is the logical inverse of while — it continues looping as long as its condition is false, and exits when the condition becomes true.

Command: vim unt.sh → chmod +x unt.sh → ./unt.sh

Script:

```

n=1

until [ $n -gt 5 ]
do
    echo "n = $n"
    n=$((n+1))
done

```

Screenshot:

```
n=1
until [ $n -gt 5 ]
do
    echo "n = $n"
    n=$((n+1))
done
:wq
```

Fig 4.13(a) — vim editor showing unt.sh with until loop

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./unt.sh
n = 1
n = 2
n = 3
n = 4
n = 5
```

Screenshot:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x unt.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./unt.sh
n = 1
n = 2
n = 3
n = 4
n = 5
shaonkhan@shaon:~/OS_Lab/LAB_06$
```

Fig 4.13(b) — Terminal output of unt.sh printing n = 1 through n = 5

Observation: The until loop ran while `[ $n -gt 5 ]` was false — that is, while n was 1 through 5. When n reached 6, the condition became true and the loop exited. The output was identical to what a while `[ $n -le 5 ]` loop would produce, confirming that until is the logical complement of while. The `n=$((n+1))` arithmetic expansion incremented the counter on each iteration.

4.14 break and continue

Purpose: To demonstrate how break terminates a loop early when a condition is met, and how continue skips the remaining commands in the current iteration and resumes at the next.

Command: vim breakcon.sh → chmod +x breakcon.sh → ./breakcon.sh

Script:

```
for i in 1 2 3 4 5
do
    if [ $i -eq 3 ]
    then
        break
    fi
    echo "i = $i"
done
```

Screenshot:

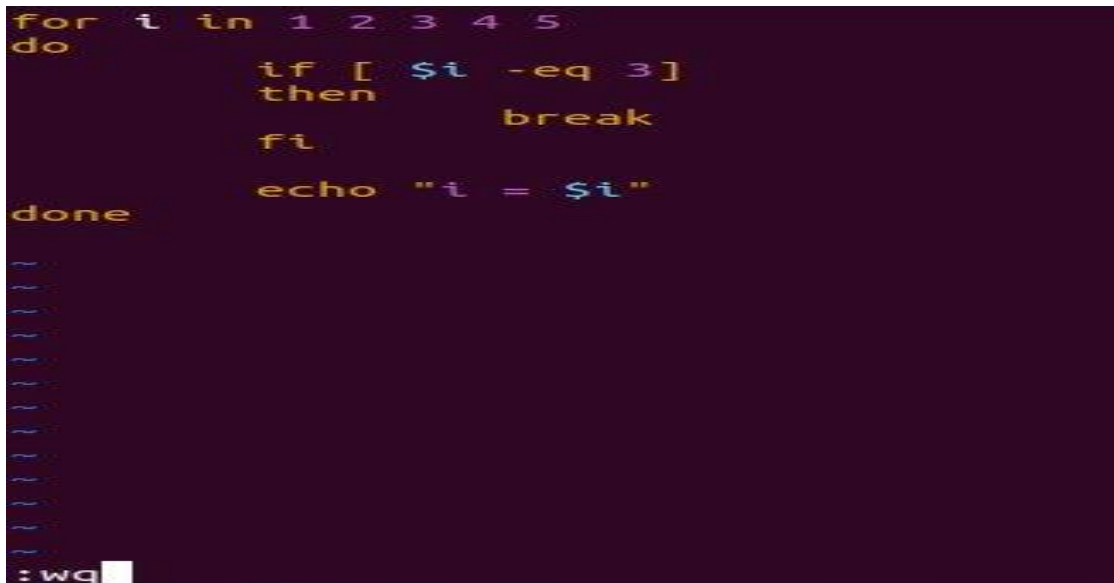


Fig 4.14(a) — vim editor showing breakcon.sh with break inside for loop

Output:

```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./breakcon.sh
./breakcon.sh: line 3: [: missing `]'
i = 1
./breakcon.sh: line 3: [: missing `]'
i = 2
./breakcon.sh: line 3: [: missing `]'
i = 3
./breakcon.sh: line 3: [: missing `]'
i = 4
./breakcon.sh: line 3: [: missing `]'
i = 5
```


Screenshot:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x breakcon.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./breakcon.sh
./breakcon.sh: line 3: [: missing `]'
i = 1
./breakcon.sh: line 3: [: missing `]'
i = 2
./breakcon.sh: line 3: [: missing `]'
i = 3
./breakcon.sh: line 3: [: missing `]'
i = 4
./breakcon.sh: line 3: [: missing `]'
i = 5

```

Fig 4.14(b) — Terminal showing breakcon.sh output with syntax warning on each iteration

Observation: The terminal output shows a "[: missing `]" warning on each iteration because the closing bracket of the if condition was missing ([\$i -eq 3] was written without the closing] on the same token). Despite the warning, Bash continued executing and printed i = 1 through i = 5, meaning the break condition was not correctly evaluated and the loop ran to completion. This highlighted that missing brackets in Bash test expressions cause runtime warnings and incorrect conditional behaviour, demonstrating the importance of exact syntax in shell scripting.

4.15 Arithmetic Expansion \$(())

Purpose: To perform integer arithmetic within a shell script using the \$(()) arithmetic expansion syntax, covering addition, subtraction, multiplication, integer division, modulo, and exponentiation.

Command: vim arith.sh → chmod +x arith.sh → ./arith.sh

Script:

```

a=10
b=3

echo "Addition: $((a+b))"
echo "Substraction: $((a-b))"
echo "Multiplication: $((a*b))"
echo "Division: $((a/b))"
echo "Modulus: $((a%b))"
echo "Exponent: $((a**b))"

```

Screenshot:

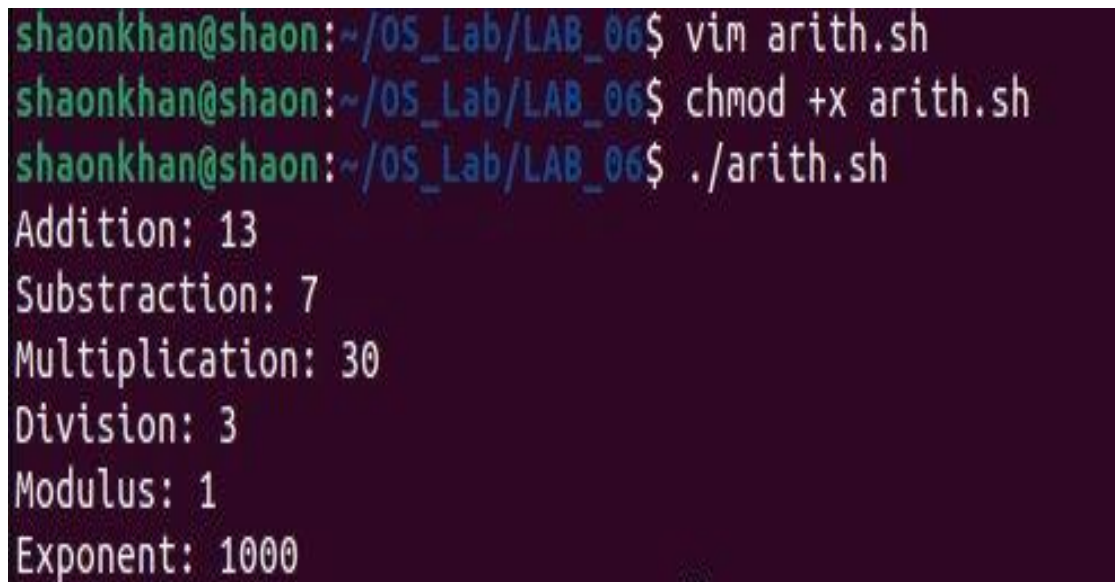

```
a=10
b=3

echo "Addition: $((a+b))"
echo "Substraction: $((a-b))"
echo "Multiplication: $((a*b))"
echo "Division: $((a/b))"
echo "Modulus: $((a%b))"
echo "Exponent: $((a**b))"
```

Fig 4.15(a) — vim editor showing arith.sh with all arithmetic operators

Output:

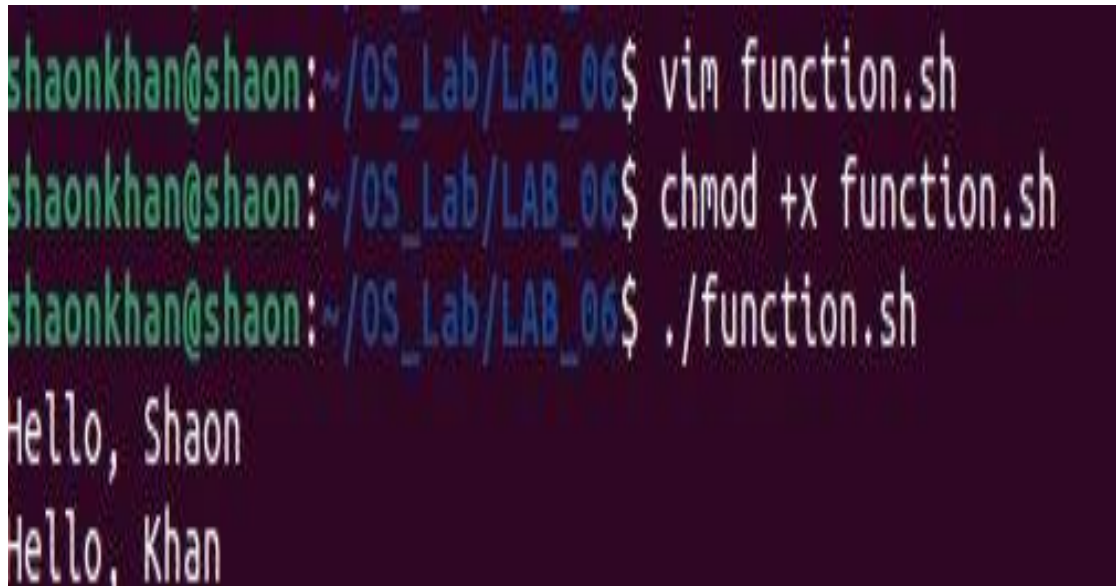
```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./arith.sh
Addition: 13
Substraction: 7
Multiplication: 30
Division: 3
Modulus: 1
Exponent: 1000
```

Screenshot:


```
shaonkhan@shaon:~/OS_Lab/LAB_06$ vim arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x arith.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./arith.sh
Addition: 13
Substraction: 7
Multiplication: 30
Division: 3
Modulus: 1
Exponent: 1000
```

Fig 4.15(b) — Terminal output of arith.sh showing all six arithmetic results

Observation: All six operators produced correct results: $10+3=13$, $10-3=7$, $10\times 3=30$, $10/3=3$ (integer division, truncated), $10\%3=1$ (remainder), and $10^{**}3=1000$. The $\$(())$ syntax is evaluated natively by Bash without any external tools. Division truncates the decimal portion since Bash supports integers only — $10/3$ gives 3, not 3.33. For floating-point results, the `bc` utility would be needed.

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim function.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x function.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./function.sh
Hello, Shaon
Hello, Khan

```

Fig 4.16(b) — Terminal output showing greet() called with two different arguments

Observation: The greet() function was defined once and called twice with different arguments. Inside the function, \$1 refers to the first argument passed at the call site — 'Shaon' in the first call and 'Khan' in the second. This confirms that functions in Bash are reusable code blocks and that positional parameters inside a function refer to the function's own arguments, not the script's arguments.

4.17 return Statement and \$? (Exit Status)

Purpose: To demonstrate the return statement inside a function, which sets the function's exit status code, and the \$? special variable, which retrieves that code to communicate a computed result back to the caller.

Command: vim return.sh → chmod +x return.sh → ./return.sh

Script:

```

add() {
    sum=$(( $1 + $2 ))
    return $sum
}

add 5 3

echo "Sum = $?"

```

Screenshot:


```

add() {
    sum=$(( $1 + $2 ))
    return $sum
}

add 5 3

echo "Sum = $?"

:wq

```

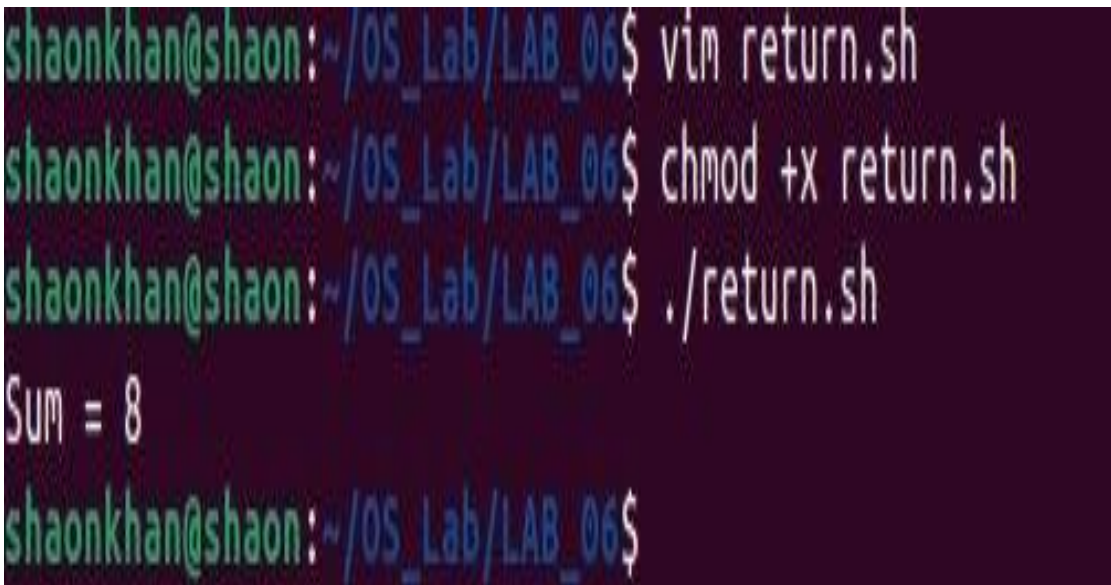
Fig 4.17(a) — vim editor showing return.sh with add() and \$? usage

Output:

```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./return.sh
Sum = 8

```

Screenshot:


```

shaonkhan@shaon:~/OS_Lab/LAB_06$ vim return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ chmod +x return.sh
shaonkhan@shaon:~/OS_Lab/LAB_06$ ./return.sh
Sum = 8
shaonkhan@shaon:~/OS_Lab/LAB_06$

```

Fig 4.17(b) — Terminal output of return.sh showing Sum = 8 via \$?

Observation: The add() function computed 5+3=8 and passed that value to return. Immediately after calling add 5 3, \$? held the return value 8, which was then printed. This demonstrates using return and \$? as a lightweight mechanism to pass small integer values from a function to the caller. It should be noted that return is limited to values 0–255 as it is designed for exit status codes; for larger or non-integer results, echoing from the function and using command substitution is the standard approach.

5. Discussion

This laboratory session provided comprehensive hands-on experience with Bash shell scripting. Seventeen subsections covered the full range of fundamental scripting concepts, and several important observations emerged from the practical work.

Script Creation and Execution: All scripts were created using vim and executed via `./script.sh` after applying `chmod +x`. The deliberate `chmod -x` experiment in subsection 4.1 produced a 'Permission denied' error, confirming that the execute bit is mandatory for `./execution` but not for `sh script.sh`. The `while.sh` syntax error (missing spaces in the test condition) and the `breakcon.sh` bracket error were valuable debugging experiences, demonstrating that Bash test expressions are highly sensitive to spacing and correct bracket syntax.

Variables and Environment Variables: User-defined variables (`name`, `num`, `count`) required no type declaration and were assigned without spaces around `=`. Environment variables (`$HOME`, `$SHELL`) were available automatically without any declaration, set by the OS at login and inherited by the script process. Both categories were accessed using the `$` prefix within double quotes, which enables variable expansion at runtime.

Output Formatting: `echo -e` and `printf` produced identical multi-line output by interpreting `\n`, confirming that `echo` without `-e` treats escape sequences as literal text. `echo -n` suppressed newlines in `for.sh` to print all numbers on a single line. `printf` is preferred for tabular output due to its format specifier support.

Command Substitution: `$(date)` captured a live timestamp including Bangladesh Standard Time (+06), demonstrating that command substitution creates dynamic, system-aware scripts. The output changed on every execution, unlike hardcoded strings.

User Input: `read` captured the full input line including spaces, as verified by testing with 'Shaon' and 'Shaon Khan'. This makes it suitable for collecting multi-word data such as full names.

Conditional Logic: The progression from `if` → `if-else` → `if-elif-else` was clearly demonstrated. The grading script confirmed that `elif` conditions are evaluated sequentially and only the first match executes. The case statement provided a clean pattern-matching alternative, handling the `y/n` input elegantly with the `*` wildcard catching all unrecognised input.

Loops: The `for` loop (list-based and C-style), `while`, and `until` loops were all successfully demonstrated. The `while.sh` syntax error highlighted the importance of correct spacing in Bash conditionals. The `until` loop produced results identical to an equivalent `while` loop but with the inverted condition, confirming they are logical complements. The `break` experiment in `breakcon.sh` revealed a common syntax pitfall — a missing closing bracket — that caused warnings on every iteration without terminating the script, demonstrating that Bash is partially tolerant of test expression errors.

Arithmetic: All six arithmetic operators (`+`, `-`, `*`, `/`, `%`, `**`) worked correctly inside `$(( ))`. Integer division truncated `10/3` to 3, and exponentiation correctly computed `10**3=1000`. The `$(( ))` syntax required no external tools, confirming it as the native Bash integer arithmetic mechanism.

Functions: The `greet()` function confirmed that `$1` inside a function refers to the function's own argument. The `return/$?` pattern in `return.sh` successfully passed the computed sum (8) back to the caller, though the 0–255 limitation of `return` means this technique is only suitable for small integers. For general data passing, `echo` combined with command substitution is the recommended approach.

6. Conclusion

This laboratory session on Introduction to Shell Scripting (Lab No. 06, ICT-3102) successfully covered all major Bash scripting concepts through seventeen practical subsections. Starting from a single-line welcome script and progressing through variables, environment variables, formatted output, command substitution, user input, all conditional constructs, four loop types, arithmetic expansion, and functions with return values, the session provided a complete and coherent introduction to shell scripting.

Several practical lessons were reinforced through real errors encountered during the session. The `chmod -x` experiment confirmed the execute bit requirement. The `while.sh` syntax error demonstrated Bash's sensitivity to spacing inside `[ ]` test conditions. The `breakcon.sh` missing-bracket issue showed that malformed conditions produce runtime warnings and incorrect behaviour rather than terminating the script. These debugging experiences are as valuable as the successful experiments, building the diagnostic skills essential for real-world scripting work.

The arithmetic experiments confirmed that `$(( ))` handles all standard integer operations natively, including exponentiation, while the function experiments established that `$1/$2` positional parameters are scoped to the function and that `return/$?` provides a lightweight integer return channel. The `until` loop was demonstrated to be the logical inverse of `while`, and the `case` statement proved more readable than chained `if-elif` blocks for fixed-value matching.

Overall, this session has built a strong, practical foundation in Bash shell scripting. The skills acquired — script creation, permission management, variable handling, input/output, conditional logic, iterative constructs, arithmetic, and modular functions — are directly applicable to system administration, task automation, and software development in any Unix/Linux environment.